

بخش ۱: توابع (Functions)

۱. هسته (Core)

1. `core.fn_validate_coupon`: وظیفه بررسی اعتبار یک کد تخفیف را بر عهده دارد. مواردی مانند فعال بودن کوپن، نگذشتن تاریخ انقضا، پر نشدن سقف دفعات مصرف، رعایت حداقل مبلغ سفارش و عدم استفاده تکراری توسط همان کاربر را چک می‌کند و یک جواب بله/خیر برمی‌گرداند.
2. `core.fn_calculate_distance_km`: مشابه تابع غذاسنجی، فاصله بین دو مختصات جغرافیایی (طول و عرض) را بر اساس فرمول هورسین با در نظر گرفتن انحنای زمین محاسبه می‌کند. این تابع به صورت عمومی برای کل سیستم (مثل محاسبه مسافت سفر) استفاده می‌شود.
3. `core.fn_has_minimum_role_level`: سطح دسترسی یک کاربر را بررسی می‌کند. با دریافت آیدی کاربر و سطح مورد نیاز، چک می‌کند که آیا کاربر مقام کافی (Hierarchy Level) برای انجام یک عملیات خاص را دارد یا خیر.
4. `core.fn_get_active_commission_rate`: نرخ کمسیون فعال در یک لحظه مشخص را برمی‌گرداند. با گرفتن نام سرویس (مثلا تاکسی یا غذا) و تاریخ، سهم دقیق پلتفرم، راننده و رستوران را با توجه به تاریخچه تغییرات نرخ‌ها استخراج می‌کند.

۲. غذا (Food)

1. `food.fn_haversine_km`: بر اساس فرمول haversine فاصله دو نقطه در زمین را بدست می‌آورد. انحنای زمین را نیز این فرمول در نظر می‌گیرد که دقیق تر از حساب فاصله در یک صفحه صاف است. چه رستوران باشد چه کاربر فاصله هر دو نقطه ای را بر اساس موقعیت جغرافیایی محاسبه میکند.
2. `food.fn_branches_within_distance`: بر اساس شعاعی که با آن تابع را فرا میخوانیم، مکان کاربر و حداکثر شعاعی که یک رستوران میتواند غذا تحویل دهد رستوران های فعالی که در آن شعاع از کاربر قرار دارند را نشان میدهد. از این جهت مهم است که یک رستوران تا چه فاصله ای سرویس میدهد و کاربر تا چه فاصله ای مایل به دریافت غذاست.
3. `food.fn_is_branch_open`: جواب به سوال ایا رستوران در زمان و تاریخ مشخص باز است یا نه. با مقایسه ساعات تاریخ کاری با ساعت و روزی که به تابع می‌دهیم. همچنین اگر به طور کامل در یک روز رستوران بسته باشد انرا در نظر می‌گیرد.
4. `food.fn_get_effective_menu_price`: قیمت یک اینم از منوی رستوران را محاسبه میکند. یک قیمت پایه داریم که اگر تخفیف نداشتیم یا تخفیف تمام شده بود یا تخفیف شروع نشده بود همان را برمیگرداند. اگر نه با حساب تخفیف قیمت را برمیگرداند.
5. `food.fn_calculate_cart_total`: محاسبه قیمیت تمام کالای سبد خرید. کمیت هر کالا را در قیمت نهایی ضرب کرده بعد جمع میکند و برمیگرداند.
6. `food.fn_calculate_order_total`: محاسبه قیمت نهایی سفارش در لحظه ای که ثبت شده. بر خلاف تابع قبل که سبد خرید را در حال حاضر محاسبه میکرد این تابع قیمت تمام ایتم های یک سفارش با توجه به کمیت ان ها محاسبه میکند.
7. `food.fn_can_cancel_order`: با استفاده از این تابع تصمیم میگیریم ایا یک سفارش را میتوان لغو کرد یا نه. مثلا اگر سفارش دریافت شده بود، کنسل شده بود و نمیتوان انرا کنسل کرد. رستوران پس از تایید سفارش زمان اضافه تری نسبت به کاربر برای کنسل کردن دارد بدلیل اینکه ممکن است موقع پخت اتفاقی بیفتد.

۳. تاکسی (Taxi)

1. `taxi.fn_get_active_pricing_parameter`: پارامترهای قیمت‌گذاری (ورودی، قیمت هر کیلومتر و قیمت هر دقیقه) را برای یک نوع وسیله نقلیه و سرویس خاص در زمان حال (یا تاریخ دلخواه) برمی‌گرداند.
2. `taxi.fn_calculate_ride_price`: قیمت نهایی یک سفر را با استفاده از تابع قبلی محاسبه می‌کند. فرمول آن جمع کرایه پایه با (مسافت ضربدر قیمت کیلومتر) و (زمان ضربدر قیمت دقیقه) است.
3. `taxi.fn_vehicle_supports_service`: بررسی می‌کند که آیا یک خودروی خاص (مثلا موتور) اجازه سرویس‌دهی در یک دستبندی خاص (مثلا جابجایی مسافر) را دارد یا خیر.

4. taxi.fn_find_nearest_available_drivers: با استفاده از تابع مسافت‌یاب هسته، لیستی از نزدیک‌ترین رانندگان آزاد (Available) را که وسیله نقلیه آن‌ها مناسب سرویس درخواستی کاربر است پیدا کرده و بر اساس کمترین فاصله مرتب می‌کند.

بخش ۲: پروسیجرها (Stored Procedures)

۱. هسته (Core)

1. core.sp_write_log: پروسیجر استاندارد برای ثبت رویدادها و لاگ‌های امنیتی در سطح هسته سیستم. عملیات انجام شده، مقادیر قدیم و جدید و عامل تغییر را ذخیره می‌کند.
2. core.sp_register_user: ثبت‌نام کاربر جدید. علاوه بر درج اطلاعات در جدول کاربران، به صورت خودکار یک کیف پول با موجودی صفر برای او می‌سازد.
3. core.sp_charge_wallet: افزایش موجودی کیف پول یک کاربر. بررسی می‌کند مبلغ منفی یا صفر نباشد، سپس پول را اضافه کرده و یک تراکنش در جدول مربوطه ثبت می‌کند.
4. core.sp_assign_admin_role: ارتقای کاربر عادی به ادمین. کاربر را به جدول ادمین‌ها اضافه کرده و آیدی نقش (role) او را در جدول کاربران به ادمین تغییر می‌دهد.
5. core.sp_apply_coupon: کدهای تخفیف را روی سفارشات یا سفرها اعمال می‌کند. ابتدا با تابع اعتبارسنجی آن را چک می‌کند و در صورت مجاز بودن، یک واحد به تعداد استفاده‌های کوپن اضافه می‌کند.
6. core.sp_check_user_exists: برای بررسی تکراری نبودن شماره موبایل، ایمیل و نام کاربری در هنگام ثبت‌نام در فرانت‌اند استفاده می‌شود.
7. core.sp_verify_login: مدیریت ورود به سیستم. پسورد هش شده، وضعیت بلاک بودن و یا حذف شدن کاربر را بررسی می‌کند و در صورت موفقیت، آیدی کاربر را برای نشست (Session) برمی‌گرداند.
8. core.sp_get_user_profile: یک خروجی تجمیع شده از اطلاعات هویتی، سطح دسترسی و موجودی کیف پول کاربر برای نمایش در پروفایل برمی‌گرداند.
9. core.sp_get_available_coupons: لیستی از کوپن‌های فعال و دارای ظرفیت را برای نمایش به کاربران خروجی می‌دهد.

۲. غذا (Food)

1. food.sp_write_log: پروسیجر که برای ذخیره رسیدگی‌ها در لاگ اسکیمای غذا. بجای اینکه هر تابع یا تریگر خودش اینکار رو کنه با این پروسیجر به قاعده همگانی برای ثبت لاگ وضع میکنیم. دو حالت برای ثبت بررسی توی لاگ داریم. اگه توسط یک عامل انجام شده باشه ایدی عامل را قرار میدهیم به همراه توضیحات و اما بعضی چیزا عاملی ندارن و توسط خود سیستم انجام میشن که ای دی عامل ایدی ادمین سیستم میشه.
2. food.sp_add_to_cart: این پروسیجر اضافه شدن به سبد خرید کاربر را مدیریت میکند که سه قاعده را رعایت میکند. کمیت غذا باید مثبت باشد. غذا باید موجود باشد. و در اخر تمام ایتm های حال حاضر سبد خرید باید از یک شعبه رستوران باشند. با احتساب این ۳ شرط به سبد خرید چیزی اضافه میکند.
3. food.sp_clear_cart: برای پاک کردن سبد خرید از این پروسیجر استفاده میکنیم. یا کل شعبه ها را پاک میکند یا فقط یک شعبه. برای اینکه مثلا بعد از خرید یک سری غذا از یک شعبه که باید سبد خرید پاک شود اگر بلافاصله از یک شعبه دیگر چیزی خرید انرا مختل نکنیم.
4. food.sp_set_branch_schedule: اجازه به رستوران میده تا برنامه کاری خود را تعریف یا اپدیت کنند. روزی که ایجاد میشود باید یکی از روز های هفته باشد. اجازه تغییر روز یا ساعت رو هم میده.
5. food.sp_create_menu_discount: تخفیف برای یکی از ایتm های منو ایجاد میکند. یک بازه زمانی و مقدار تخفیف میخواهد. اعتبار سنجی تخفیف با تریگر انجام میشود و این پروسیجر حکم گرفتن ورودی برای ایجاد تخفیف را دارد.
6. food.sp_get_available_branches_for_customer: این پروسیجر ترکیب دو سوال در جایی که قرار داری چه شعبه هایی از نظر جغرافیایی در دسترس تو قرار دارند(تابع بخش قبل) و اینکه کدام شعبه ها با توجه به تاریخ و زمان

باز هستند است که در نهایت باعث میشود کاربر بداند توانایی سفارش از کدام رستوران ها را دارد. شعبه هایی را برمیگرداند که ترکیب پاسخ مثبت به این دو سوال را دارند.

7. `food.sp_assign_restaurant_staff`: به هر شعبه رستوران کارکنان انجا را تخصیص میدهد. دو شرط دارد. یکی اینکه نمیتوان یک کارکن را دوبار به یک رستوران تخصیص داد. همچنین چون صاحب شعبه خودکار میتواند کارکن رستوران خود باشد یا نباشد اجازه نمیدهد صاحب شعبه را تخصیص دهیم.

8. `food.sp_cancel_order`: این پروسیجر لغو یک غذا را مدیریت میکند. با تابع بخش قبل رابطه دارد اگر حالت غذا در مرحله تولید تا محصول بیش از اندازه پیش رفته باشد مثلاً اگر تحویل داده شده باشد اجازه لغو نمیدهد. اگر اجازه کنسل کردن داشته باشیم سفارش را کنسل و دلیل انرا ذکر میکند. بعد چک میکند آیا کاربر پول داده است یا نه. اگر اره پس پول را برمیگرداند و این تراکنش را ثبت میکند (در جدول تراکنش هسته).

۳. تاکسی (Taxi)

1. `taxi.sp_write_log`: مشابه بقیه اسکیمها، وظیفه لاگ کردن عملیات حیاتی و حساس در حوزه ناوگان و سفرها را بر عهده دارد.

2. `taxi.sp_register_driver`: ثبتنام کاربر به عنوان راننده تاکسی. آیدی نقش او را به راننده تغییر داده و همزمان برای او کیف پول در آمدی در اسکیمای تاکسی میسازد.

3. `taxi.sp_register_vehicle`: وسایل نقلیه متعلق به یک راننده را با پلاک و مدل ثبت کرده و وضعیت آن را فعال در نظر میگیرد.

4. `taxi.sp_create_ride_offer`: نقطه شروع یک سفر. مبدا و مقصد را گرفته، قیمت را محاسبه میکند و پیشنهاد سفر را به نزدیکترین رانندگانی که از طریق تابع هورسین پیدا شدهاند ارسال میکند.

5. `taxi.sp_respond_to_ride_offer`: وقتی رانندهای پیشنهاد را قبول یا رد میکند. اگر قبول کند، وضعیت سایر رانندگان کاندید شده را به `timeout` (منقضی شده) تغییر داده، یک رکورد سفر واقعی ایجاد کرده و وضعیت راننده را به «مشغول» تغییر میدهد.

6. `taxi.sp_update_ride_status`: ماشین حالت (State Machine) سفرها. تغییر وضعیت سفر (حرکت به مبدا، رسیدن به مقصد، تکمیل) را با اعتبارسنجی منطق تقدم و تاخر زمانی هندل میکند تا سفری بی دلیل تکمیل یا لغو نشود.

7. `taxi.sp_start_ride_to_origin / taxi.sp_start_ride_to_destination`: توابع کمکی برای ارسال راحتتر وضعیت های حرکتی به پروسیجر ماشین حالت.

8. `taxi.sp_pay_for_ride`: سیستم پرداخت میانی. پول را از کیف پول کاربر در هسته کسر کرده و تراکنش را با موفقیت ثبت میکند.

9. `taxi.sp_complete_ride`: پروسیجر تسویه نهایی. پس از رسیدن به مقصد، در صورت عدم پرداخت هزینه آن را پرداخت می کند، سپس سهم راننده را به کیف پول تاکسی او واریز کرده، سهم پلتفرم را کسر می کند و راننده را دوباره «آزاد» می کند.

10. `taxi.sp_cancel_ride`: لغو سفر. راننده را آزاد می کند و در صورتی که مسافر پیش پرداخت کرده باشد، کل مبلغ را از طریق رکورد `refund` (استرداد) به حساب او در هسته برمی گرداند.

11. `taxi.sp_rate_ride`: سیستم ثبت امتیاز دوطرفه. هم راننده می تواند به مسافر امتیاز دهد و هم بالعکس، و تضمین می کند که هر نفر فقط یکبار برای یک سفر بتواند امتیاز ثبت کند.

12. (Read APIs): مجموعه ای از پروسیجرها (مانند `sp_get_passenger_active_ride` و

`sp_get_passenger_ride_history`) که برای تغذیه دیتای فرانت اند و نمایش تاریخچه یا وضعیت فعلی سفرهای کاربران و رانندگان ساخته شده اند.

بخش ۳: تریگرها (Triggers)

۱. هسته (Core)

1. `core.trg_users_after_insert`: به محض اینکه کاربری در دیتابیس ثبت نام کند، مشخصات و زمان آن را در جدول

لاگ‌های سیستم ممیزی (Audit) می‌کند.

2. `core.trg_transactions_after_insert`: هر تراکنش مالی (شارژ، پرداخت، استرداد) که ثبت شود، بلافاصله آن را در لاگ مرکزی برای پیگیری‌های مالی و رفع اختلافات ذخیره می‌کند.
3. `core.trg_admins_after_insert`: هرگاه فردی در سیستم به عنوان ادمین ثبت شود، این تریگر به صورت اتوماتیک وارد عمل شده و سطح دسترسی (`role_id`) او را در جدول کاربران همگامسازی و آپدیت می‌کند.
4. `core.trg_coupons_after_update`: سیستم خاموش‌کننده خودکار کدهای تخفیف. هر بار که کد تخفیفی اعمال شود، چک می‌کند که آیا تعداد دفعات مصرف به سقف مجاز رسیده یا خیر. در صورت رسیدن به سقف، وضعیت کوپن را به «غیرفعال» (`is_active = 0`) تغییر می‌دهد تا دیگر قابل استفاده نباشد.

۲. غذا (Food)

1. `food.trg_menu_discounts_validate`: وقتی تخفیفی برای یک ایتم اضافه می‌شود بررسی می‌کند معتبر است یا نه تا اجازه اعمال آنرا بدهد. دو شرط دارد. باید قیمت مشخص یا درصد مشخصی کاهش یابد. مثلاً درصد بیرون ۰ تا ۱۰۰ بی معنی است. نمیتواند یک غذا در یک زمان دو تخفیف متفاوت داشته باشد. این بررسی زمانی شامل بررسی بازه تخفیف‌ها میشود تا بازه‌ها همپوشانی نداشته باشند.
2. `food.trg_branches_after_insert_create_wallet`: این تریگر وقت ساخت یک شعبه فعال میشود تا کیف پول شعبه را بسازد. برای اطمینان حاصل کردن از اینکه شعبه حتما مکانی برای واریز پول داشته باشد.
3. `food.trg_cart_items_validate`: از هر طریقی به سبد خرید چیزی اضافه شود اطمینان حاصل میکند درست است. برای یک برند و یک شعبه از آن باشد. ایتم موجود باشد. برای اطمینان حاصل کردن است در صورتی که پروسیجر اینجا درست عمل نکند.
4. `food.trg_food_orders_timestamp_autofill`: زمان اینکه یک سفارش به چه مرحله‌ای از پختن تا دریافت رسیده است را به طور خودکار ثبت میکند تا لازم نباشد هر بار دستی ثبت شود.
5. `food.trg_food_orders_status_validate`: باعث میشود یک سفارش از پختن تا دریافت هر بار وضعیتش مرحله به مرحله و روبه جلو تغییر کند. اگر به مرحله‌نهایی رسید از تغییر آن جلوگیری میکند.
6. `food.trg_food_orders_audit`: هر وقت وضعیت سفارش تغییر کرد آنرا ثبت میکند. و اگر کسی آنرا تغییر داده باشد فردی که آنرا تغییر داده هم همراه آن ثبت میکند.
7. `food.trg_food_orders_after_insert`: لحظه ثبت سفارش را ثبت میکند. اگر کسی که آنرا ثبت کرده به طور دقیق ذکر نشود کاربر را به عنوان ثبت کننده پیش فرض در نظر می‌گیرد.
8. `food.trg_branches_audit`: تغییرات وضعیت فعال بودن و ریتینگ یک شعبه را مدیریت میکند و اگر تغییر کردند آنرا ثبت میکند.
9. `food.trg_order_purchase`: تریگری که برای خرید یک سفارش است. قیمت را با توجه به تخفیفات و بقیه مواردی که در بالا گفته شد محاسبه کرده و میبیند چقدر از آنرا پرداخت نکرده ایم. انگاه قیمت را محاسبه کرده و سهم رستوران و اپلیکیشن را میدهد. اگر اولین پرداخت باشد و همچنین سفارش بیرون بر باشد از اسکیمای تاکسی درخواست یک سفر را انجام میدهد.

۳. تاکسی (Taxi)

1. `taxi.trg_vehicles_enforce_single_active`: تضمین‌کننده این قانون که یک راننده در یک لحظه فقط میتواند با یک ماشین در سیستم آنلاین باشد. اگر راننده‌ای ماشین جدیدی را فعال کند، این تریگر فوراً ماشین قبلی او را غیرفعال می‌کند.
2. `taxi.trg_drivers_after_insert` و `taxi.trg_vehicles_after_insert`: هرگونه ورود راننده جدید یا ثبت ماشین جدید در ناوگان را به صورت قالب JSON در لاگ‌های امنیتی ذخیره می‌کنند.
3. `taxi.trg_rides_after_insert`: آخرین سد امنیتی پیش از آغاز سفر. بررسی می‌کند که آیا وسیله نقلیه‌ای که سفر را قبول کرده اصلاً مجوز چنین سفری را دارد یا نه (مثلاً موتور مسافر سوار نکرده باشد). اگر مغایرت داشته باشد تراکنش را برمی‌گرداند (Rollback).
4. `taxi.trg_rides_log_status_change` و `taxi.trg_rides_log_rating`: تاریخچه‌ی دقیقی از هرگونه تغییر

وضعیت در سفرها (مثل کنسل شدن) و ثبت امتیازات را با جزئیات مقادیر قدیم و جدید در لاگ‌ها ثبت می‌کنند.

5. taxi.trg_rides_update_metrics: موتور گیمیفیکیشن و رتبه‌بندی. به محض اینکه مسافر یا راننده‌ای امتیاز دریافت کند، این تریگر بیدار شده و به صورت پویا (Dynamic) معدل امتیازات (Average Rating) و رتبه‌ی فعالیت (Activity Points) آن‌ها را در دیتابیس بازنویسی می‌کند.